

Advanced Python Operators: Logic, Precedence, and Properties

This guide extends the basic understanding of bitwise and boolean operators by incorporating logic simplification, operator precedence, and formal verification using truth tables.

1. Comprehensive Operator Precedence

When multiple operators appear in a single expression, Python evaluates them in a specific hierarchy. Use parentheses `()` to override this order.

Rank	Operator	Description
1	<code>()</code>	Parentheses (Grouping)
2	<code>**</code>	Exponentiation
3	<code>+x</code> , <code>-x</code> , <code>~x</code>	Unary Plus/Minus, Bitwise NOT
4	<code>*</code> , <code>/</code> , <code>//</code> , <code>%</code>	Multiplication, Division, Floor Div, Modulo
5	<code>+</code> , <code>-</code>	Addition, Subtraction
6	<code><<</code> , <code>>></code>	Bitwise Left and Right Shifts
7	<code>&</code>	Bitwise AND

8	<code>^</code>	Bitwise XOR
9	<code> </code>	Bitwise OR
10	<code>==</code> , <code>!=</code> , <code>></code> , <code><</code> , <code>>=</code> , <code><=</code> , <code>is</code> , <code>in</code>	Comparisons, Identity, Membership
11	<code>not</code>	Logical NOT
12	<code>and</code>	Logical AND
13	<code>or</code>	Logical OR
14	<code>=</code> , <code>+=</code> , <code>-=</code> , etc.	Assignments (Right-to-Left associativity)

2. Boolean Algebra Properties

These properties allow us to simplify complex logic statements. Let A , B , and C be boolean values.

Fundamental Properties

- **Closure:** The result of a boolean operation is always a boolean (`True` or `False`).
- **Commutative:** $A \text{ and } B = B \text{ and } A$ | $A \text{ or } B = B \text{ or } A$
- **Associative:** $(A \text{ and } B) \text{ and } C = A \text{ and } (B \text{ and } C)$
- **Distributive:**
 - $A \text{ and } (B \text{ or } C) = (A \text{ and } B) \text{ or } (A \text{ and } C)$
 - $A \text{ or } (B \text{ and } C) = (A \text{ or } B) \text{ and } (A \text{ or } C)$ (Unlike regular math, OR distributes over AND!)

Simplification & Absorption

- Identity: $A \text{ and } \text{True} = A \mid A \text{ or } \text{False} = A$
- Null (Domination): $A \text{ and } \text{False} = \text{False} \mid A \text{ or } \text{True} = \text{True}$
- Idempotent: $A \text{ and } A = A \mid A \text{ or } A = A$
- Inverse (Complement): $A \text{ and } (\text{not } A) = \text{False} \mid A \text{ or } (\text{not } A) = \text{True}$
- Absorption: $A \text{ or } (A \text{ and } B) = A \mid A \text{ and } (A \text{ or } B) = A$
- Double Negation: $\text{not } (\text{not } A) = A$
- De Morgan's Laws:
 - $\text{not } (A \text{ and } B) = (\text{not } A) \text{ or } (\text{not } B)$
 - $\text{not } (A \text{ or } B) = (\text{not } A) \text{ and } (\text{not } B)$

3. Verifying Logic with Truth Tables

A Truth Table lists all possible inputs to prove if two expressions are equal.

Example: Verifying $A \text{ and } (A \text{ or } B) = A$ (Absorption)

A	B	(A or B)	A and (A or B)	Result
T	T	T	T	Matches A
T	F	T	T	Matches A
F	T	T	F	Matches A
F	F	F	F	Matches A

F

F

F

F

Matches A

4. Homework Exercises (Complete Set)

Perform these manually before verifying with Python.

Set A: Bitwise & Basic Logic

1. What is $12 \& 7$?
2. What is $15 | 10$?
3. What is $20 \gg 3$?
4. Calculate 10 .
5. What is $8 \ll 2$?
6. What is $64 \gg 3$?
7. Evaluate $(5 \& 3) | 4$.
8. Evaluate $(10 \gg 1) \& 7$.
9. What is the result of $1 \wedge 1 \wedge 1$?
10. Calculate (-5) .
11. Evaluate: `True or False and False`
12. Evaluate: `not (True and False)`
13. Evaluate: `(5 > 3) and (2 == 2)`
14. What does `0 or 5` return in Python?
15. What does `10 and 20` return in Python?

16. Evaluate: `not ("apple" == "orange")`
17. If `a = True` and `b = True`, what is `a != b`?
18. Perform an 8-bit circular left shift on 128 (10000000_2) by 1.
19. What is `bool(10) ^ bool(0)`?
20. Evaluate: `(7 & 1) == 1` and `(8 >> 1) == 4`

Set B: Precedence & Property Simplification

21. Evaluate: `True or False and not True`
22. Evaluate: `(23 > 42) and (36 > 12) or (39 <= 39)`
23. Evaluate: `10 + 5 << 1` (Remember precedence!)
24. Simplify: `A or (not A and B)`
25. Simplify: `(A and B) or (A and not B)`
26. Simplify: `A or (False and B)`
27. Simplify: `not (not A or B)` using De Morgan's.
28. Evaluate: `5 * 2 ** 2 & 15`
29. Evaluate: `not (15 // 3 == 5 and 7 % 2 == 0)`
30. Simplify: `(A or B) and (not A or B)`
31. Simplify: `(A and B and C) or (A and B and not C)`
32. Evaluate: `~5 & 7`
33. Evaluate: `1 << 3 | 1 << 1`
34. Simplify: `A and (A or B)` (Name the property).

35. Evaluate: `(True or False) and (not True or True)`
36. Simplify: `A or (A and B) or (B and False)`
37. Perform an 8-bit circular right shift on 1 (00000001_2) by 1.
38. Evaluate: `10 > 5 > 2` (Chained comparison)
39. Evaluate: `not (5 < 2) or 10 // 2 == 5`